

RecruitVerse

AI-Powered Resume Screening & Candidate Ranking System

The Complete Project Guide & Interview Preparation Handbook

PGCP Big Data Analytics · CDAC Innovation Park, Pune
Final Project · From Level 0 to Interview-Ready

What's Inside

Read this start to finish once. Then re-read Sections 8 and 9 the night before your presentation.

1. **The 30-Second Pitch** — how to describe the project in one breath
2. **Level 0: What Is This Project, Really?** — the idea in plain words
3. **Key Concepts Explained Simply** — embeddings, semantic search, ATS, NLP
4. **The Complete Workflow** — what happens end to end
5. **How the Score Is Calculated** — the math, demystified
6. **The Technology Stack** — every tool and why we chose it
7. **Architecture & Code Structure** — how the code is organised
8. **Module-by-Module Deep Dive** — what each file does
9. **The Datasets** — what they are and how they're used
10. **Interview Questions & Answers** — 50+ Q&As, basic to hard
11. **Anticipating the Tough Questions** — closing every loophole
12. **Limitations & Honest Weaknesses** — what to admit gracefully
13. **Future Scope** — where the project can go
14. **Glossary** — every term, one line each

RecruitVerse — Project Guide

This handbook explains the entire project in the simplest possible language, then arms all five team members with answers to every question an examiner could ask. Nothing is assumed. If a term appears, it is explained.

1. The 30-Second Pitch

When someone says "So, what is your project?" — this is your answer. Memorise the shape of it.

SAY THIS

"RecruitVerse is an **AI-powered Applicant Tracking System**. A recruiter uploads a job description and a batch of resumes. The system reads every resume, compares each one against the job using **semantic similarity and skill matching**, and produces a **ranked shortlist** of candidates — best to worst. Crucially, it **explains every score**, so the recruiter knows exactly why a candidate ranked where they did. It also uses a local AI model to write summaries, interview questions and hiring recommendations — but the AI never decides the score; the score is calculated deterministically so it's fair and reproducible."

That single paragraph answers *what*, *who it's for*, *how it works*, and pre-empts the most common trap question ("does the AI just make up the score?"). Everything else in this guide expands on it.

2. Level 0: What Is This Project, Really?

Imagine you posted a job for a Data Engineer. **200 people apply**. Each sends a resume — some PDFs, some Word files. Reading all 200 by hand would take days, and by resume #150 you're tired and inconsistent. You might reject a great candidate just because they wrote "PySpark" instead of "Spark."

RecruitVerse automates exactly this. Think of it as a **smart assistant that has read all 200 resumes in seconds** and hands you a sorted list: "Here are your top candidates, here's why each scored what they did, and here's what each one is missing."

What is an "ATS"?

ATS = Applicant Tracking System. It's the category of software big companies use to manage job applications. Real examples: Greenhouse, Workday, Naukri's recruiter tools. Our project is a focused, AI-enhanced ATS built around the single most valuable task: **screening and ranking**.

The one-sentence version

CORE IDEA

Turn a pile of resumes + one job description into a **ranked, explained shortlist** — using AI to understand *meaning*, not just keywords.

Why is this a "Big Data Analytics" project?

Three reasons, and an examiner may ask this directly:

- **Volume:** the backend knowledge base holds tens of thousands of resumes (54,000 in one dataset alone). Processing and searching that is a data-at-scale problem.
- **Unstructured data:** resumes are free text — messy, inconsistent. Turning them into structured, searchable data is a classic analytics challenge.
- **Machine learning / NLP:** we use embedding models (a form of ML) to understand text meaning, and vector search to retrieve similar candidates — core techniques in modern data science.

3. Key Concepts Explained Simply

If you understand these five ideas, you understand the whole project. Each is explained as if to someone who has never heard the term.

3.1 What is an "embedding"?

A computer can't understand words directly — it understands numbers. An **embedding** is a way of turning a piece of text into a list of numbers (a **vector**) that captures its *meaning*.

The magic: texts with similar meaning get similar numbers. So "built data pipelines" and "developed ETL workflows" end up with vectors that are close together, *even though they share no common words*. That's how the system understands that a candidate who wrote one has experience relevant to a job asking for the other.

ANALOGY

Think of every possible sentence as a point on a giant map. Sentences about cooking cluster in one region; sentences about data engineering cluster in another. An embedding is simply the **map coordinates** of a sentence. Two sentences that mean similar things sit close together on the map.

3.2 What is "semantic similarity"?

Semantic means "related to meaning." **Semantic similarity** is a score of how close two pieces of text are *in meaning*. We compute it as the **cosine similarity** between two embedding vectors — a number that's high when the vectors point in the same direction (similar meaning) and low when they don't.

This is the opposite of a plain keyword search, which only matches exact words. Semantic similarity is why our system is smarter than a simple "Ctrl+F for Python" filter.

3.3 What are "Sentence-Transformers"?

It's the specific open-source tool (a Python library) we use to create embeddings. It provides pre-trained AI models that have already learned, from millions of sentences, how to map text to meaning-vectors. We use a model called `all-MiniLM-L6-v2` — "Mini" because it's small and fast enough to run on a normal laptop, which matters for a live demo.

3.4 What is "vector search" (semantic search)?

Once every resume is an embedding (a vector), searching becomes geometry. You type "Data Engineers with Spark," we turn *that* into a vector too, and then we find the resume-vectors **closest** to it on the map. Those are your best matches — ranked by closeness. This is called **vector search**, and it powers our Candidate Search page.

3.5 What is an "LLM" and what is "Ollama"?

LLM = Large Language Model — the kind of AI that writes human-like text (ChatGPT is one). **Ollama** is a tool that lets you run an LLM **locally on your own computer**, for free and privately, instead of calling a paid cloud service.

In our project the LLM's job is strictly **writing helpful text**: resume summaries, interview questions, hiring recommendations. **It never calculates the score**. Remember this line — it's the single most important distinction in the whole project (Section 5 explains why).

4. The Complete Workflow

Here is exactly what happens, step by step, when a recruiter uses RecruitVerse. This mirrors the eight pages in the app's sidebar.

Step	What happens
1. Job Description	Recruiter pastes or uploads a JD (PDF/DOCX/TXT). The system extracts required skills, preferred skills, experience needed, and education — and displays them before any matching begins.
2. Resume Upload	Recruiter drops in 4–20 resumes at once. A progress bar shows parsing status per file.
3. Parsing	Each resume's text is extracted and structured into: name, email, phone, skills, experience, education, certifications, projects, previous companies, and a summary.
4. Semantic Matching	Each resume and the JD are turned into embeddings; cosine similarity gives a meaning-based match score.
5. Skill Matching	The JD's skills are compared to each resume's skills, producing three lists: matched, missing, and additional skills.
6. Ranking	A weighted final score (skills + semantic + experience) is computed for every candidate, and they're sorted best-to-worst with medals for the top three.
7. Candidate Details	Clicking a candidate opens a full profile: score breakdown, skill lists, and a ✓/✗ explanation for the score.
8. AI Insights	On demand, the local LLM writes a summary, strengths/weaknesses, a hiring recommendation, and tailored interview questions.

DEMO TIP

In your live demo, follow these steps **in order** and narrate each one. "Now I paste the job description... notice it extracted the required skills automatically... now I upload six resumes... and here's the ranked shortlist with the reasoning." A narrated walkthrough is far more convincing than clicking silently.

5. How the Score Is Calculated (The Most Important Section)

Examiners love this question because it separates students who *built* the project from those who just *ran* it. Learn this cold.

5.1 The formula

THE FINAL MATCH SCORE

Final Score = (0.50 × Skill Match)
+ (0.30 × Semantic Similarity)
+ (0.20 × Experience Relevance)

Each of the three parts is a number from 0 to 100. They are blended using fixed weights (50%, 30%, 20%) into one final percentage. Those weights live in one place in the code (`settings.RANKING_WEIGHTS`) so they're easy to tune and defend.

5.2 What each component means

Component (weight)	How it's computed
Skill Match (50%)	Of all the skills the JD lists as <i>required</i> , what fraction does this resume have? If the JD needs 8 skills and the candidate has 6 of them, that's 75%. Skills are matched with synonym handling (so "PySpark" counts as "Spark").
Semantic Similarity (30%)	The whole resume and the whole JD are each turned into an embedding vector. We take the cosine similarity between them and rescale it to a 0–100 score. This captures overall relevance beyond individual skill keywords.
Experience Relevance (20%)	Candidate's years of experience vs. the years the JD asks for. Meeting or exceeding the requirement = 100. Falling short scores proportionally (e.g. 2 years when 4 are needed = 50).

5.3 Why these weights?

Skills are weighted highest (50%) because, for most technical roles, *having the required skills is the strongest single signal* of fit. Semantic similarity (30%) adds nuance — it rewards relevant experience phrased in ways a skill list misses. Experience (20%) matters but shouldn't dominate; a brilliant 3-year candidate can beat a mediocre 6-year one. **These weights are a design choice, not a law** — and you can say exactly that if challenged. They're configurable in one line.

5.4 THE GOLDEN RULE — the AI never decides the score

MEMORISE THIS

The score is **100% deterministic**. It comes only from embeddings, cosine similarity, skill counting, and the weighted formula above. The **LLM (Ollama) is never involved in scoring or ranking**. The LLM only writes *explanatory text* — summaries, interview questions, recommendations.

Why did we design it this way? Three strong reasons — give all three if asked:

- Reproducibility:** the same resume and JD always produce the same score. An LLM can give different answers each time it's asked; a hiring decision can't be that flaky.
- Fairness & defensibility:** because the score is a transparent formula, we can show *exactly* why a candidate scored what they did. "The AI felt like it" is not an acceptable reason to reject someone.
- No hallucination risk:** LLMs sometimes invent facts. We never let an invented fact affect a real ranking. The LLM's text is clearly labelled as assistance, and it even falls back to rule-based text if the LLM is switched off.

5.5 A worked example

JD requires 8 skills, 4 years experience. Candidate Priya has 6 of the 8 skills, 6 years experience, and her resume reads as

strongly relevant.

Component	Value	Weighted
Skill Match (6 of 8)	75	$75 \times 0.50 = 37.5$
Semantic Similarity	87	$87 \times 0.30 = 26.1$
Experience ($6 \geq 4$)	100	$100 \times 0.20 = 20.0$
Final Score		83.6%

And the explanation panel would show: ✓ Python matched, ✓ SQL matched, ... ✗ Airflow missing, ✓ Experience requirement satisfied. Fully transparent.

6. The Technology Stack

For each tool: what it is, and **why we chose it** (examiners always ask "why this and not something else?").

Technology	What it is & why we used it
Python	The whole project is Python — the standard language for data science and ML, with the richest ecosystem of libraries for text processing and embeddings.
Streamlit	A Python framework for building web apps with no HTML/JavaScript. Chosen because it let us build a clean, interactive recruiter UI quickly and stay in one language. Ideal for data apps and demos.
FastAPI	A modern, fast Python web-API framework. Provides an optional REST API so the same screening engine can be called by other software, not just our UI.
Sentence-Transformers	The library that generates embeddings for semantic matching. Chosen because it's open-source, runs offline, and the <code>all-MiniLM-L6-v2</code> model is small and fast enough for a live demo while still being accurate.
Ollama	Runs a Large Language Model locally for the AI text features. Chosen over a cloud API for privacy (resumes are sensitive data), zero cost, and offline operation.
PostgreSQL	A robust open-source relational database for optional persistence of candidates and jobs. The app also runs fully in-memory, so the DB is a bonus, not a requirement.
PyMuPDF / python-docx	Libraries to extract text from PDF and Word resumes respectively.
Docker	Packages the app and database into containers so the whole system runs with one command on any machine — solving "it works on my laptop but not yours."
pandas / NumPy	Standard data-handling libraries — used for reading dataset CSVs and vector math.

IF ASKED: "WHY STREAMLIT AND NOT DJANGO/FLASK?"

Django and Flask are general web frameworks that need HTML, CSS and JavaScript for the front end. Streamlit is purpose-built for data apps — a few lines of Python produce interactive tables, file uploaders and charts. For a data-analytics project with a small team and a demo focus, it's dramatically faster to build and maintain. FastAPI is still included for anyone who needs a raw programmatic API.

7. Architecture & Code Structure

The project follows a **clean, layered architecture**: the user interface is kept separate from the business logic, which is kept separate from the AI and data layers. This means each part can be understood, tested and changed independently.

7.1 The three layers

- Presentation layer** (`src/ui/` + `app.py`): the Streamlit pages. This layer only *displays* things and collects input — it contains no scoring logic.
- Logic layer** (`src/parser` , `src/matching` , `src/ranking` , `src/pipeline` , `src/explainability`): all the real work — parsing, embeddings, scoring, ranking. This is deterministic and fully testable.
- Support layers** (`src/ai` , `src/knowledge_base` , `src/retrieval` , `src/database`): the optional LLM features, the dataset-backed search index, and optional persistence.

7.2 Folder map

app.py	Streamlit entry point (routing only)
src/	
parser/	file_extractor, resume_parser, jd_parser
matching/	embedding_matcher (semantic), skill_matcher
ranking/	scoring_engine, candidate_ranker, analytics
explainability/	score_explainer (the ✓/✗ reasons)
ai/	ollama_client, insights (all LLM text)
pipeline/	screening_pipeline (ties it all together)
knowledge_base/	dataset_loader (builds the search index)
retrieval/	search_candidates (semantic search)
ui/	theme, state, and the 8 pages
api/	FastAPI routes (optional)
data/	skills dictionary + the 3 datasets
sql/schema.sql	optional database schema
tests/	automated tests

7.3 The "pipeline" — the heart of the system

One file, `screening_pipeline.py`, orchestrates the whole deterministic flow so the UI and the API share the exact same logic and can never drift apart. Its job, in plain terms:

1. Take one parsed JD and a list of parsed resumes.
2. Embed the JD once and all resumes in a single batch (efficient).
3. For each resume: compute skill match, semantic score, experience score.
4. Blend them with the weighted formula into a final score.
5. Attach the ✓/✗ explanation.
6. Sort everyone best-to-worst and return the ranked list.

8. Module-by-Module Deep Dive

What each important file does, in one or two sentences. If an examiner points at any part of the code, you'll know its job.

Module	Responsibility
<code>file_extractor.py</code>	Extracts plain text from PDF, DOCX or TXT — one function used by both resume and JD parsing.
<code>resume_parser.py</code>	Turns raw resume text into a structured profile: name, email, phone, skills, experience, education, certifications, projects, companies, summary.
<code>jd_parser.py</code>	Parses a job description and splits skills into required vs preferred (by detecting "nice to have" sections).
<code>skill_extractor.py</code>	Scans text for known skills using a dictionary of 102 skills across 13 categories, with word-boundary matching.
<code>embedding_matcher.py</code>	The semantic engine: loads the Sentence-Transformers model, turns text into vectors, computes cosine similarity. The ONLY place similarity is calculated.
<code>skill_matcher.py</code>	Compares resume skills to JD skills; outputs matched, missing and additional skill lists plus a skill-coverage score.
<code>scoring_engine.py</code>	Computes the experience score and the final weighted blend.
<code>candidate_ranker.py</code>	Sorts candidates by final score and assigns rank numbers and medals.
<code>score_explainer.py</code>	Produces the structured ✓/✗ reason list — works with no LLM at all.
<code>ollama_client.py</code>	A thin client that talks to the local LLM; returns nothing gracefully if Ollama is offline.
<code>insights.py</code>	All AI text features (summary, strengths, recommendation, interview questions, tips, JD summary) — each with a rule-based fallback.
<code>dataset_loader.py</code>	Loads the three datasets, embeds a sample of each, and builds the vector index used by Candidate Search.
<code>screening_pipeline.py</code>	Orchestrates parse → match → score → rank so the UI and API share one code path.

9. The Datasets

Three datasets act as a **backend knowledge base**. The critical point examiners test: **recruiters never see or browse these directly**. They exist so semantic Candidate Search has a large corpus to search over.

Dataset	What it is & how it's used
Relational 54K	~54,000 structured people records spread across linked CSV tables (people, skills, education, experience). Used to build searchable candidate profiles for benchmarking and semantic retrieval.
Synthetic 10K	10,000 machine-generated resumes covering many job roles. Clean and labelled — perfect for testing that ranking and retrieval behave correctly.
LiveCareer / Real	Real-world resume text exported to CSV and grouped by job category. Adds authentic, messy variety that synthetic data lacks.

How they become searchable

A background script (`dataset_loader.py`) reads a sample of each dataset, converts every resume into an embedding vector once, and saves them as a **vector index** on disk. Later, when a recruiter types a natural-language query, we embed the query and find the closest vectors. This is the same principle behind modern semantic search engines.

LIKELY QUESTION: "WHY SAMPLE AND NOT USE ALL 54,000?"

Embedding tens of thousands of resumes takes time and memory. For a recruiter-facing search demo, a few thousand well-chosen samples give fast, relevant results. The sample size is a single configurable setting — we can raise it for a full offline rebuild without changing any code.

10. Interview Questions & Answers

Grouped from basic to advanced. Every team member should be able to answer the Basic and Intermediate sets; at least two of you should own the Advanced set.

Basic

Q1. What does your project do in one line?

It screens and ranks job candidates by comparing their resumes against a job description using AI-based semantic matching and skill analysis, and explains every score.

Q2. Who is the user?

A recruiter or hiring manager who needs to shortlist candidates quickly and fairly from a large number of applications.

Q3. What problem does it solve?

Manual resume screening is slow, inconsistent, biased, and unexplainable. Our system makes it fast, consistent, and fully transparent.

Q4. What file formats can it read?

PDF, DOCX and TXT — for both resumes and job descriptions.

Q5. What are the main pages of the app?

Dashboard, Job Description, Resume Upload, Candidate Ranking, Candidate Details, Candidate Search, AI Insights, and Settings.

Intermediate

Q6. How exactly is the match score calculated?

It's a weighted blend: 50% skill match + 30% semantic similarity + 20% experience relevance, each scored 0–100. Skill

match is the fraction of required JD skills present in the resume; semantic similarity is the cosine similarity of the resume and JD embeddings; experience compares candidate years to required years.

Q7. What is an embedding, in your own words?

A list of numbers representing the meaning of a piece of text, such that texts with similar meaning have similar numbers. It lets the computer compare meaning, not just exact words.

Q8. What is cosine similarity?

A measure of the angle between two vectors. If two embedding vectors point in nearly the same direction, cosine similarity is close to 1 (very similar meaning); if perpendicular, close to 0. We use it to score how relevant a resume is to a JD.

Q9. Which embedding model do you use and why?

all-MiniLM-L6-v2 from Sentence-Transformers. It's small, fast on CPU, runs offline, and is accurate enough for resume-to-JD matching — a good balance for a live demo on a laptop.

Q10. What's the difference between semantic search and keyword search?

Keyword search matches exact words. Semantic search matches meaning, so "built ETL pipelines" can match a query about "data engineering" even with no shared words. Ours is semantic.

Q11. What are "matched," "missing," and "additional" skills?

Matched = skills the JD wants and the candidate has. Missing = required skills the candidate lacks. Additional = skills the candidate has that the JD didn't ask for (potential bonus value).

Q12. Where does Ollama / the LLM fit in?

Only in the AI Insights layer — writing summaries, strengths/weaknesses, hiring recommendations and interview questions. It never computes the score or the ranking.

Q13. What happens if Ollama isn't running?

The app still works fully. Every AI feature has a deterministic, rule-based fallback that generates reasonable text from the structured data, so no feature breaks.

Q14. How do you handle skill synonyms like "PySpark" vs "Spark"?

We normalise skills through a synonym map before matching, so common variants (ML→Machine Learning, JS→JavaScript, PySpark→Spark) are treated as the same skill.

Q15. Is a database required to run it?

No. It runs fully in-memory using session state. PostgreSQL is optional, for persisting candidates and jobs between sessions.

Advanced

Q16. Why did you make the scoring deterministic instead of letting the LLM score?

Three reasons: reproducibility (same input → same score every time), fairness and defensibility (we can show exactly why a score was given), and no hallucination risk (an LLM can invent facts; we never let that affect a real hiring decision). The LLM assists with language, not judgement.

Q17. How does the pipeline stay efficient with 20 resumes?

We embed the JD once and all resumes in a single batched call to the model, rather than embedding each pair separately. That turns roughly 20 model calls into effectively one batch, which is much faster.

Q18. How would this scale to millions of resumes?

Currently the vector index is held in memory for the demo. To scale, we'd move to a dedicated vector database like FAISS or PostgreSQL's pgvector, which do approximate nearest-neighbour search in sub-second time over millions of vectors. That's listed in our future scope.

Q19. How do you avoid bias in screening?

The deterministic, transparent formula is the first defence — every decision is inspectable. We deliberately score on skills, semantic relevance and experience, not on personal attributes. A formal bias-and-fairness audit module is in our future scope.

Q20. What's the architecture pattern?

A layered / clean architecture: presentation (UI) is separated from the deterministic logic layer, which is separated from support layers (AI, data, persistence). The pipeline module is the single orchestration point shared by the UI and the API.

Q21. How is the JD's "required vs preferred" split decided?

The JD parser looks for marker phrases like "preferred," "nice to have," or "bonus." Skills before that marker are required; skills after it are preferred. If no marker exists, everything is treated as required.

Q22. Why rescale the cosine similarity?

Raw cosine similarity for sentence embeddings tends to sit in a narrow band (roughly 0.2–0.9 for related text) rather than spanning 0–1, so multiplying by 100 directly would make good matches look deceptively low to a recruiter. We rescale against a practical floor so the numbers read intuitively.

Q23. How did you test the system?

Automated tests cover the deterministic core end to end — skill extraction, skill matching, scoring math, ranking order, parsing, and the full pipeline (with the embedding step stubbed so tests run fast). All tests pass.

Q24. What was the biggest engineering challenge?

Refactoring a large existing codebase into a focused ATS — deciding what to reuse (skill dictionary, parsers, DB layer) versus rebuild (real semantic matching, weighted scoring, the UI), while keeping everything working. And ensuring the app degrades gracefully when optional pieces (Ollama, database) are absent.

11. Anticipating the Tough Questions (Closing Every Loophole)

These are the "gotcha" questions designed to find gaps. Prepared answers below.

"Isn't this just a keyword matcher with extra steps?"

No. A keyword matcher only checks exact words. We add genuine semantic understanding via embeddings, so we catch relevant candidates who phrase things differently — plus we produce explainable, weighted rankings, not a binary keyword hit/miss.

"The AI could be wrong — how can you trust the ranking?"

The ranking doesn't come from the AI. It's a deterministic formula over skills, semantic similarity and experience. The AI only writes descriptive text, and even that has rule-based fallbacks. So the ranking is fully trustworthy and reproducible regardless of the LLM.

"What if a resume is badly formatted or a scanned image?"

We handle PDF, DOCX and TXT text extraction. A purely scanned image PDF with no text layer is a known limitation — the honest answer is that OCR (optical character recognition) would be needed, which we've noted in future scope. For text-based resumes it works well.

"Your name/experience extraction uses simple rules — isn't that fragile?"

Yes, parsing free-form resumes is inherently imperfect, and we're upfront about that. We use robust heuristics (regex for

email/phone/experience, a 102-skill dictionary, section detection) that work on well-structured resumes. Improving this with a trained NER model is in future scope. Importantly, the semantic score works on the whole resume text, so it's resilient even when field extraction is imperfect.

"Why only 102 skills? Real resumes have thousands of skills."

The dictionary is a curated, extensible starting set across 13 categories relevant to tech/data roles. It's a single JSON file — adding skills is trivial. And the semantic matching layer catches relevance even for skills not in the dictionary, because it compares whole-text meaning.

"How is this 'Big Data' if you only rank 20 resumes at a time?"

The recruiter-facing ranking handles a working batch, but the backend knowledge base holds tens of thousands of resumes (54K in one dataset), all embedded into a vector index for semantic search. The data volume, the unstructured-text processing, and the ML/NLP techniques are what make it a big-data analytics project.

"Did you train your own model?"

We use a pre-trained embedding model (transfer learning) rather than training from scratch — which is standard, efficient practice. Training a domain-specific embedding model on tech resumes is a clear future enhancement. We were honest about using pre-trained models rather than claiming we trained one.

"What stops two identical-looking candidates getting different scores?"

Nothing does — that's the point. Identical inputs give identical scores because the pipeline is deterministic. There's no randomness anywhere in scoring.

"Show me where in the code the score is computed."

Point to `scoring_engine.py` (the weighted blend and experience score), `skill_matcher.py` (skill coverage), and `embedding_matcher.py` (semantic similarity). The pipeline that calls them all in order is `screening_pipeline.py`.

12. Limitations & Honest Weaknesses

Admitting limitations **gracefully** earns more marks than pretending there are none. Frame each as "known, and here's the path to fix it."

- **Resume parsing is heuristic.** Free-text field extraction (name, companies) can miss on unusually formatted resumes. Mitigation: a trained NER model.
- **No OCR.** Scanned-image PDFs with no text layer won't parse. Mitigation: add an OCR step (e.g. Tesseract).
- **In-memory search index.** Fine for a demo; wouldn't scale to millions live. Mitigation: FAISS or pgvector.
- **Skill dictionary is finite.** Semantic matching compensates, but a larger/auto-expanding skill list would help. Mitigation: mine skills from the datasets automatically.
- **LLM quality depends on the local model.** Smaller local models write simpler text. Mitigation: allow plugging in a larger model; the scoring is unaffected either way.
- **No authentication / multi-user yet.** It's a single-session tool. Mitigation: add accounts and role-based access for real deployment.

13. Future Scope

- **Bias & fairness auditing** — detect and report demographic bias so screening stays equitable and compliant.
- **Vector database (FAISS / pgvector)** — scale semantic search from thousands to millions of resumes with sub-second retrieval.
- **ATS & job-board integrations** — pull candidates automatically from LinkedIn, Naukri, Greenhouse.
- **Fine-tuned domain embeddings** — train the embedding model specifically on technical resumes for sharper matching.
- **Recruiter analytics** — time-to-hire, funnel metrics and pipeline dashboards.
- **Authentication & multi-tenant SaaS** — user accounts, role-based access, per-company data isolation.
- **OCR support** — read scanned-image resumes.
- **Better parsing via NER** — a trained model for names, companies, dates and titles.

14. Glossary

Term	One-line meaning
ATS	Applicant Tracking System — software to screen and manage job applications.
Embedding	A list of numbers (vector) that represents the meaning of text.
Vector	An ordered list of numbers; here, the numeric form of a piece of text.
Cosine similarity	A 0–1 score of how similar two vectors are in direction (meaning).
Semantic similarity	How close two texts are in meaning, computed via cosine similarity of embeddings.
Semantic / vector search	Finding the most meaning-similar items by comparing embedding vectors.
Sentence-Transformers	The Python library/model we use to create embeddings.
NLP	Natural Language Processing — teaching computers to work with human text.
LLM	Large Language Model — an AI that generates human-like text.
Ollama	A tool to run an LLM locally, offline and free.
Deterministic	Always produces the same output for the same input; no randomness.
Transfer learning	Using a pre-trained model instead of training from scratch.
Explainability	Being able to show clearly why the system produced a given result.
Pipeline	A sequence of processing steps run in order (parse → match → score → rank).
FAISS / pgvector	Tools for fast similarity search over very large sets of vectors.
NER	Named Entity Recognition — an ML technique to find names, dates, companies in text.
OCR	Optical Character Recognition — extracting text from images/scans.

FINAL ADVICE BEFORE YOU PRESENT
1) Every member should be able to give the 30-second pitch. 2) Split ownership: assign the scoring math, the architecture, the datasets, the AI layer, and the demo to different members so no single question stumps the whole team. 3) When you don't know something, say "*That's a great extension — it's in our future scope*" rather than guessing. Confidence + honesty beats bluffing every time. You built a genuinely solid project. Present it like you own it.